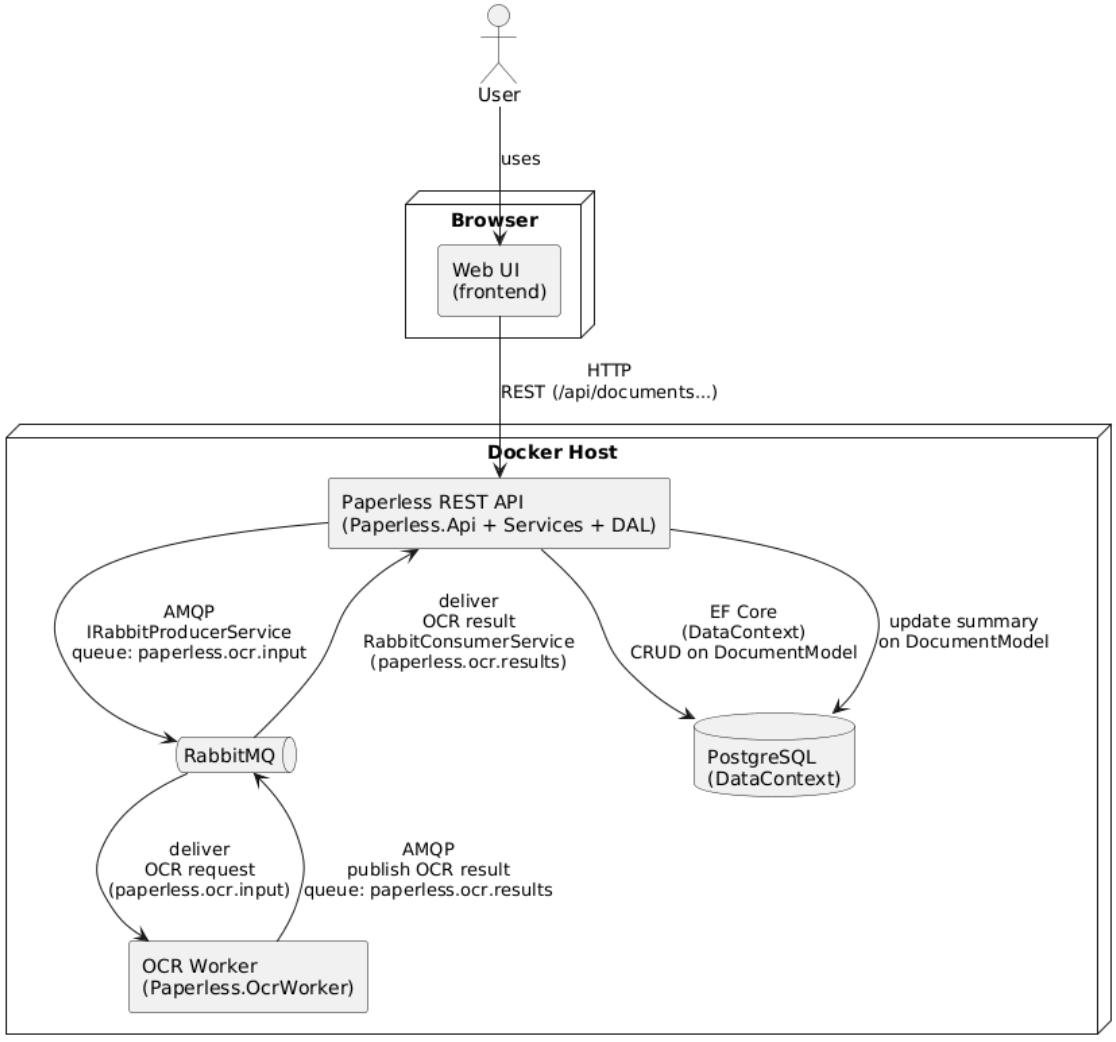
Architecture (Sprints 1–3)

1. **Components (as of Sprint 3):**

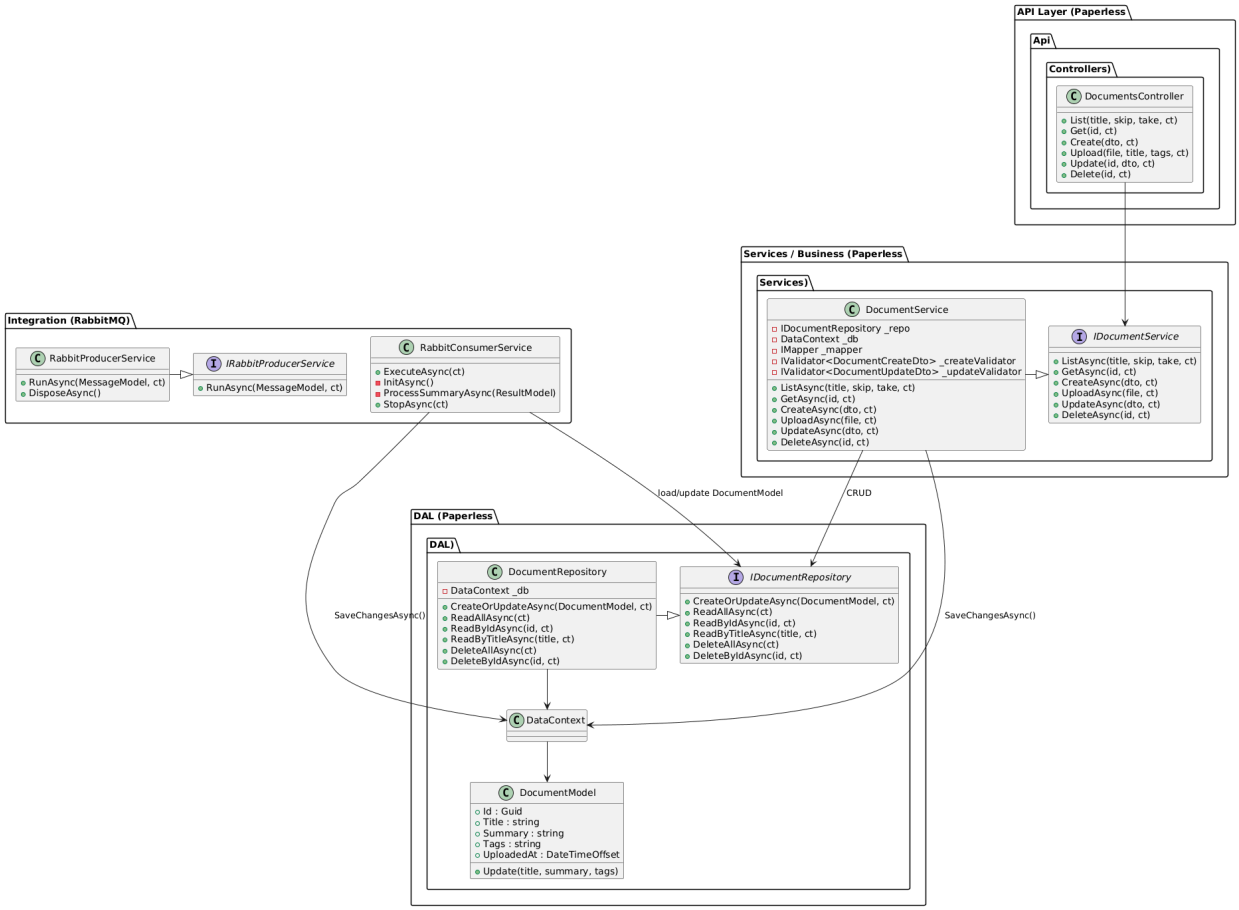
- Frontend (Quasar app in frontend):
  - Runs in container paperless-frontend behind nginx web server
- REST API (paperless backend):
  - Handles document upload and retrieval
  - Runs in container paperless-backend
- PostgreSQL:
  - Persists document metadata and other entities
  - Runs in container paperless-postgres
- RabbitMQ:
  - Message broker for document processing
  - Runs in container paperless-rabbitmq
- Worker Service:
  - Subscribes to queue and logs OCR work (placeholder for real OCR later)
  - Runs in container paperless-ocrworker



2. **Layered Architecture (Backend)**

- API Layer
  - Controllers: DocumentController
- Application / Business Layer
  - Services:
    - DocumentService
- Infrastructure / Data Access Layer
  - Repositories: DocumentRepository
  - DbContext: DataContext
- Integration Layer
  - Queue publisher: RabbitProducerService
  - Queue consumer: RabbitConsumerService

3. **Class Diagram (Core Domain)**



4. **Sequence Diagram – Upload Document**
   Sequence for POST /api/documents/upload:
   Actors: **Frontend → API → Repository/DB → Queue → Worker**

   1. Frontend sends POST /api/documents with file/metadata.
   2. API validates request.
   3. API calls DocumentService.CreateDocument(dto).
   4. Service calls DocumentRepository.Add(document).
   5. Repository saves via DbContext to PostgreSQL.
   6. Service / API publishes a message via DocumentQueuePublisher.

7. RabbitMQ receives message.
8. Worker service receives message and writes log entry.
9. Worker service publishes result back
10. RabbitMQ receives message.
11. REST API receives message with generated summary
12. API calls Repo to update DB entry with summary